# Day 18 Task: Linux permission exercise

**Linux File Permissions and Security**

Linux is an open source operating system widely used in servers, networking, cloud computing, and cybersecurity. It provides security mechanisms that control access to files and directories through user accounts, groups, file ownership and permissions.

Incorrect file permissions can create security vulnerabilities by allowing unauthorized users to read, modify or execute files. Proper permission management helps protect sensitive information and maintain system security.

**Permission Groups**

Every file and directory in Linux has three main permission groups.

Owner: The user who owns the file or directory. Owner permissions determine what that user can do with the resource.

Group: The users who belong to the group associated with the file or directory. Group permissions control their access.

Others: All other users on the system who are neither the owner nor members of the associated group.

**Permission Types**

Linux provides three basic permission types.

Read (r): Allows users to read a file's contents. For directories, it allows users to list directory entries.

Write (w): Allows users to modify a file's contents. For directories, it permits creating, deleting, or renaming entries, subject to other applicable permissions and restrictions.

Execute (x): Allows users to execute a file when appropriate. For directories, it allows users to traverse the directory.

No permission (-): Indicates that the corresponding permission is not granted.

**Understanding Linux Permission Notation**

The ls -l command displays file permissions and ownership information.

Example:

-rwxr-xr-- 1 alice developers 1200 Oct 19 script.sh

The permission string rwxr-xr-- contains three groups:

- rwx — the owner can read, write, and execute.
- r-x — the group can read and execute.
- r-- — others can only read.

The first character indicates the file type. A hyphen (-) represents a regular file, while d represents a directory.

Numeric Permission Values

Linux permissions can be represented using numbers.

| Permission | Numeric value |
|---|---|
| Read (r) | 4 |
| Write (w) | 2 |
| Execute (x) | 1 |
| No permission | 0 |

The values are added for each permission group.

For example, chmod 755 script.sh assigns read, write, and execute permissions to the owner and read and execute permissions to the group and others.

The resulting permissions are rwxr-xr-x.

Another example is chmod 600 private.txt, which gives the owner read and write permissions while denying access permissions to the group and others.

Changing File Permissions with chmod

The chmod command changes file and directory permissions.

Symbolic method:

chmod u+x script.sh

Adds execute permission for the owner.

chmod g-w report.txt

Removes write permission from the group.

chmod o-r report.txt

Removes read permission from others.

Numeric method:

chmod 644 report.txt

Gives the owner read and write permissions, while the group and others receive read permission only.

File Ownership

Linux assigns an owner and an associated group to every file and directory. Ownership determines which user and group permission rules apply.

The chown command changes file ownership.

sudo chown alice report.txt

Changes the file owner to alice.

sudo chown alice:developers report.txt

Changes the owner to alice and the associated group to developers.

The chgrp command can also be used to change the group associated with a file.

Important Linux Permission Commands

| Command | Purpose |
|---|---|
| ls -l | Displays file permissions and ownership |
| chmod | Changes file permissions |
| chown | Changes file ownership |
| chgrp | Changes group ownership |
| whoami | Displays the current username |
| id | Displays user and group IDs |
| groups | Displays the groups associated with a user |
| sudo | Runs an authorized command with elevated privileges |

Linux Security Best Practices

Linux security can be improved by assigning permissions according to the needs of each user and application. Sensitive files should not be accessible to everyone, and unnecessary write or execute permissions should be removed.

Administrators should avoid using the root account for routine tasks and use sudo only when elevated privileges are required. User accounts and group memberships should be reviewed regularly, and unused accounts should be disabled or removed according to organizational procedures.

Applying the principle of least privilege helps reduce the risk of unauthorized access, accidental modifications, and damage caused by compromised accounts.

Linux file permissions are an important part of system security. Understanding permission groups, permission types, ownership, and commands such as chmod and chown helps administrators control access to system resources. Following the principle of least privilege and reviewing permissions regularly can reduce security risks and protect sensitive information.